Core 90 - Heaps and top-k

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6	1
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Heaps and top-k | 7 questions | tap a title to jump to it

M	1. Top-K Frequent in Large Logs	1Point3Acres	2026-07-19
M	2. Sliding-Window Mean After Removing the Largest K Values	PracHub	2026-09-01
M	3. Active Communication Top Users	1Point3Acres	2026-09-22
M	4. Streaming Median (LC 295) with Follow-ups	1Point3Acres	2026-08-03
M	5. Rearrange Tasks With Cooldown	PracHub	2026-06-03
M	6. Batch and Streaming Log Deduplication	1Point3Acres	2026-07-23
H	7. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant	1Point3Acres	2026-10-09

STUDY PATH: question 32 of 167 (Core 90)

Priority score 6.35 | Heaps and top-k | asked 93 days ago | frequency Medium

MEDIUM | IN APP LC 347 | 1Point3Acres | phone-screen, onsite-coding | 1 of 7

★ TOP 75: rank 29 in the Medium and Hard priority list

★ Top-K Frequent in Large Logs

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 58.8** | **30 DAYS 41.1** | **PRIORITY 21/228**

ABOUT THIS QUESTION

Topic: Core 90 - Heaps and top-k

Optimal answer from: your app's interview approach (LeetCode #347)

LeetCode match: LC 347 Top K Frequent Elements

Frequency: Medium | Last asked: 2026-07-19 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/top-k-frequent-from-logs

QUESTION

EXAMPLES

```
Input:  chatLog = ["A says hi", "B says hi", "A says
    bye", "C says hi", "A says yo"], k=2
Counts: A=3, B=1, C=1
Output: [A, B]  // or [A, C] — ties allowed
```

APPROACH

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

HOW TO THINK ABOUT IT

Count the speakers first; a log line is just a way to read the user id. Then you only need the K largest counts: a heap of size K does that without sorting all users. For logs spread over many machines the local top K are not enough on their own, so say what extra information you send.

Basis: LeetCode 347, Top K Frequent Elements (here the key is the speaker in each chat line).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(chat_log, k):
    names = [l.split()[0] for l in chat_log]
    counts = Counter(names)
    def rank(name):
        return (-counts[name], name)
    return sorted(counts, key=rank)[:k]
```

BRUTE FORCE

TIME	SPACE
O(n + d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct speakers, O(d log d), when only K are wanted; a size-K heap is O(d log k), and a stream needs a counter sketch instead of exact counts.

OPTIMAL SOLUTION · Count, then keep a size-K heap

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you do it across many machines when the log does not fit on one?

Count per shard and send each shard's local top-K to a coordinator. That is not exact in general (a user can be 11th everywhere yet first overall), so for an exact answer use the threshold algorithm: gather the local top-K, compute a global lower bound, ask every shard for all users above bound / numShards, and merge. If an approximate answer is acceptable, merge count-min sketches (they add) and keep a heap of the K largest estimates.

FOLLOW-UP 2 What if the log arrives as an unbounded stream?

Use the Misra-Gries or Space-Saving summary with m counters. Any user with more than n / (m + 1) messages is guaranteed to be in the summary, and a second pass (or a count-min sketch for the estimate) can confirm the real counts. **Cost:** O(1) per message with O(m) memory for m counters (Misra-Gries), instead of O(d) memory for the exact counts.

FOLLOW-UP 3 Which approach would you choose and why?

A heap is the safe default: simple, O(n + d log k) and streaming-friendly. Quickselect wins when k is a large fraction of d and you need all K but not their order. Sorting is fine only when d is small. Say that out loud, with the complexities, because the interviewer is grading the trade-off discussion.

FOLLOW-UP 4 The log is a stream and memory is limited.

Misra-Gries keeps m counters; any user with more than n / (m + 1) messages is guaranteed to remain in the summary (checked above). A second pass, or a count-min sketch, then gives the real counts of the survivors.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Top-K Frequent in Large Logs
#
# Examples: Input: chatLog = ["A says hi", "B says hi", "A
# says bye", "C says hi", "A says yo"], k=2 Counts: A=3,
# B=1, C=1 Output: [A, B] // or [A, C] — ties allowed
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you do it across many machines when the log
# does not fit on one?
# 2. What if the log arrives as an unbounded stream?
# 3. Which approach would you choose and why?
# 4. The log is a stream and memory is limited.
#
# INTERVIEW NOTES
# Count the speakers first; a log line is just a way to
# read the user id. Then you only need the K largest
# counts: a heap of size K does that without sorting all
# users. For logs spread over many machines the local top
# K are not enough on their own, so say what extra
# information you send.
#
# BRUTE FORCE (say it first): Count everything, then sort
# all users
# Count the speakers and sort every distinct user by
# (count, name). Correct and the oracle; sorting all d
# users does more work than keeping the best K. Time O(n +
# d log d). Space O(d).


import heapq
from collections import Counter


# BRUTE FORCE (say it first): Count everything, then sort
# all users | time O(n + d log d)
def top_k_speakers_sort(chat_log, k):
    c = Counter(
        line.split(" ", 1)[0] for line in chat_log
    )
    return sorted(c, key=lambda u: (-c[u], u))[:k]


# HELPERS for the base question
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def rank_key(item):
    # more messages first; ties by name
    return (-item[1], item[0])


# BASE QUESTION (optimal): Count, then keep a size-K heap
# Idea: Parse the speaker from each line and count in a
# hash map. nsmallest keeps a heap of size k, so it never
# sorts all d distinct users. Ties break by user name so
# the output is deterministic.
def top_k_speakers(chat_log, k):
    counts = Counter(speaker(line) for line in chat_log)
    # a heap of size k: O(n + d log k)
    items = counts.items()
    best = heapq.nsmallest(k, items, key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 1: How do you do it across many machines when
# the log does not fit on one?
# Answer: Count per shard and send each shard's local
# top-K to a coordinator.
def merge_shards(shard_logs, k):
    # exact answer: add the full counts of every shard
    total = Counter()
    for log in shard_logs:
        total.update(speaker(line) for line in log)
    best = heapq.nsmallest(k, total.items(), key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 2: What if the log arrives as an unbounded
# stream?
# Answer: Use the Misra-Gries or Space-Saving summary with
# m counters. Any user with more than n / (m + 1) messages
# is guaranteed to be in the summary, and a second pass
# (or a count-min sketch for the estimate) can confirm the
# real counts.
# FOLLOW-UP 4: The log is a stream and memory is limited.
# Answer: Misra-Gries keeps m counters; any user with more
# than n / (m + 1) messages is guaranteed to remain in the
# summary (checked above). A second pass, or a count-min
# sketch, then gives the real counts of the survivors.
class MisraGries:
    """m counters: a user with over n / (m + 1) of the
    n messages always stays in the summary."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, user):
        if user in self.counts:
            self.counts[user] += 1
        elif len(self.counts) < self.m:
            self.counts[user] = 1
        else:
            # no room: every counter pays one
            for u in list(self.counts):
                self.counts[u] -= 1
                if self.counts[u] == 0:
                    del self.counts[u]

    def candidates(self):
        return set(self.counts)


if __name__ == "__main__":
    log = ["A says hi", "B says hi", "A says bye",
           "C says hi", "A says yo"]
    assert top_k_speakers(log, 2) == ["A", "B"]
    assert top_k_speakers(log, 5) == ["A", "B", "C"]
    assert top_k_speakers([], 3) == []
    # two shards that each rank someone else first
    s1 = ["A says x", "B says x"]
    s2 = ["B says y", "B says z"]
    assert merge_shards([s1, s2], 1) == ["B"]
    mg = MisraGries(2)
    for u in "AAABBC":
        mg.add(u)
    # A has 3 of 6 messages (> 6 / 3), so it must survive
    assert "A" in mg.candidates()
```

NOTES FROM CANDIDATES

The interviewer typically wants a discussion of the three implementations + complexity trade-off, not just one working solution.
For the distributed follow-up, candidates that mention count-min sketch and topK per shard -> global merge got positive feedback.
The same family surfaces as "Top-K IPs", "Top-K chatters", and "Top-K words"; the technique is interchangeable.
A recent variant uses a gigabyte-scale IP list and explicitly requires exact time and space complexity analysis.

PREPARATION

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

STUDY PATH: question 33 of 167 (Core 90)

Priority score 6.19 | Heaps and top-k | asked 49 days ago

MEDIUM | PracHub | Technical Screen | 2 of 7

Sliding-Window Mean After Removing the Largest K Values

POSTED 2026-09-01 | 49 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 29.8 30 DAYS 0 PRIORITY 126/228

ABOUT THIS QUESTION

Topic: Core 90 - Heaps and top-k

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #480 Sliding Window Median

LeetCode match: LC 480 Sliding Window Median (order statistics in a window)

Posted on PracHub: 2026-09-01 (updated 2026-09-23)

Original: https://prachub.com/coding-questions/sliding-window-mean-after-removing-the-largest-k-values

QUESTION

For every contiguous window of a fixed size, remove the largest **k** values by multiplicity and compute the mean of the remaining values. Process sliding updates incrementally rather than sorting each complete window again. Implement **`trimmed_window_means(nums: int[], window: int, k: int) -> string[]`**. Return means in window order as reduced fractions **`numerator/denominator`**, using a positive denominator. This exact fraction encoding is an explicit practice representation that avoids cross-language floating-point rounding differences.

Constraints & Assumptions

- **`1 <= len(nums) <= 200000`**, **`1 <= window <= len(nums)`**, and **`0 <= k < window`**.
- Values range from -1,000,000 through 1,000,000. Use a sufficiently wide integer for window sums.
- Remove exactly **k** occurrences, not **k** distinct values. Equal values on the partition boundary may be represented in either group as long as the multiset counts are correct.
- For **`k == 0`**, retain the whole window. A window that would retain zero values is excluded by the input contract.
- Reduce each fraction by the greatest common divisor of the absolute numerator and denominator. Zero is **`0/1`**; an integer mean such as 5 is **`5/1`**.
- Aim for **`O(n log window)`** processing with **`O(window)`** maintained state, apart from the returned results.

Examples

```
nums = [1,9,2,8], window = 3, k = 1 result = ["3/2","5/1"]
```

```
nums = [-1,1,3], window = 2, k = 0 result = ["0/1","2/1"]
```

QUESTION

Explain how insertions, removals, duplicate values, and rebalancing preserve the retained sum. For the reported large-data/stream follow-up, describe which state must remain in memory and why a new window does not require a full sort.

```
Maintain a size and an ordering invariant Partition the window into the removed largest values and the retained values. Keep their sizes correct and ensure no retained value is larger than a removed value.
```

APPROACH

Keep the window in two structures (top-k multiset and the rest) with running sums, using two heaps with lazy deletion or a balanced BST; mean = rest_sum / (w - k). O(n log w).

HOW TO THINK ABOUT IT

The mean needs two sums: the whole window and the sum of its K largest values. Keep the window in a sorted container; the K largest are its last K elements. When one value enters or leaves, only one element crosses the boundary between 'kept' and 'removed', so the sum of the K largest can be updated in O(1) after an O(log) search.

Basis: LeetCode 480 (Sliding Window Median) style order statistics; the mean of the window after dropping exactly K occurrences of the largest values, returned as reduced fractions.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, w, k):
    means = []
    for i in range(len(nums) - w + 1):
        window = sorted(nums[i:i + w])
        kept = window[:w - k]  # drop k largest
        total = sum(kept)
        means.append(Fraction(total, len(kept)))
    return means
```

TIME	SPACE
O(n * w log w)	O(w)

WHY IT IS SLOW

Each window is sorted from scratch, O(n * w log w). A sorted list updated by one insertion and one removal, with the sum of the top k adjusted at the boundary, avoids it.

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

TIME	SPACE
O(n * (log w + w) in memmove, O(n log w) with a balanced tree)	O(w)

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the update O(log w) in the worst case.

A Python list insert shifts elements (fast in practice, O(w) in theory). A balanced tree or a Fenwick tree over compressed values gives O(log w) insert, delete and 'sum of the top K' via a descent.

FOLLOW-UP 2 Remove the smallest K instead, or both ends (trimmed mean).

Maintain the sum of the K smallest as well (the first K elements of the same sorted list); the mean of the middle values is (total - top - bottom) / (w - 2K).

FOLLOW-UP 3 Duplicates on the boundary.

Removing exactly K occurrences, not K distinct values, is what the positional approach does: elements equal to the boundary value are interchangeable, so which copy sits in the top K does not matter for the sum. **Cost:** unchanged: O(w) per update for list insertion (O(log w) with a balanced tree).

FOLLOW-UP 4 Trim the smallest K as well (a trimmed mean).

Maintain the sum of the K smallest alongside the K largest with the mirror-image boundary updates; the mean of the middle values is (total - top - bottom) / (w - 2K). Checked against sorting every window.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Sliding-Window Mean After Removing the
# Largest K Values
#
# For every contiguous window of a fixed size, remove the
# largest k values by multiplicity and compute the mean of
# the remaining values. Process sliding updates
# incrementally rather than sorting each complete window
# again. Implement trimmed_window_means(nums: int[],
# window: int, k: int) -> string[]. Return means in window
# order as reduced fractions numerator/denominator, using
# a positive denominator. This exact fraction encoding is
# an explicit practice representation that avoids cross-
# language floating-point rounding differences. ###
# Constraints & Assumptions - 1 <= len(nums) <= 200000, 1
# <= window <= len(nums), and 0 <= k < window. - Values
# range from -1,000,000 through 1,000,000. Use a
# sufficiently wide integer for window sums. - Remove
# exactly k occurrences, not k distinct values.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make the update O(log w) in the worst case.
# 2. Remove the smallest K instead, or both ends (trimmed
# mean).
# 3. Duplicates on the boundary.
# 4. Trim the smallest K as well (a trimmed mean).
#
# INTERVIEW NOTES
# The mean needs two sums: the whole window and the sum of
# its K largest values. Keep the window in a sorted
# container; the K largest are its last K elements. When
# one value enters or leaves, only one element crosses the
# boundary between 'kept' and 'removed', so the sum of the
# K largest can be updated in O(1) after an O(log) search.
#
# BRUTE FORCE (say it first): Sort every window and sum
# the kept values
# For each window sort a copy, drop the largest k values,
# and compute the exact fraction. Correct and the oracle;
# every window is sorted from scratch. Time O(n * w log
# w). Space O(w).


from bisect import bisect_left, bisect_right
from fractions import Fraction


# HELPERS for the brute force
def as_text(fr):
    return f"{fr.numerator}/{fr.denominator}"


def brute(nums, window, k, both=False):
    # sort every window: the check used by the demo
    out = []
    for i in range(len(nums) - window + 1):
        part = sorted(nums[i:i + window])
        part = part[k:-k] if both and k else part
        part = part[:len(part) - k] if not both else part
        mean = Fraction(sum(part), len(part))
        out.append(as_text(mean))
    return out


# HELPERS for the base question
# FOLLOW-UP 4: Trim the smallest K as well (a trimmed
# mean).
# Answer: Maintain the sum of the K smallest alongside the
# K largest with the mirror-image boundary updates; the
# mean of the middle values is (total - top - bottom) / (w
# - 2K). Checked against sorting every window.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class Window:
    """A sorted window that keeps the sum of its k largest
    and k smallest values while values come and go."""

    def __init__(self, k):
        self.k, self.s = k, []
        self.total = self.top = self.bottom = 0

    def insert(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_right(s, x)
        s.insert(p, x)
        self.total += x
        # top-k region is the last k places (new list)
        if old_len < k:
            self.top += x
        elif p >= old_len + 1 - k:
            # x entered it: the value at old_len - k left
            # it
            self.top += x - s[old_len - k]
```

```
        # bottom-k region is the first k places
        if old_len < k:
            self.bottom += x
        elif p < k:
            self.bottom += x - s[k]

    def remove(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_left(s, x)
        self.total -= x
        if old_len <= k:
            self.top -= x
            self.bottom -= x
        else:
            if p >= old_len - k:
                # the value just below the region joins it
                self.top += s[old_len - k - 1] - x
            if p < k:
                self.bottom += s[k] - x
        del s[p]


# BASE QUESTION (optimal): Sorted window with an
# incrementally maintained top-K sum
# Idea: Keep the window values in a sorted list, the total
# window sum, and top = the sum of its K largest elements.
def trimmed_window_means(nums, window, k, both=False):
    # mean of each window after dropping the k largest
    # (and, with both=True, the k smallest as well)
    w, out = Window(k), []
    for i, x in enumerate(nums):
        w.insert(x)
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - (2 * k if both else k)
            drop = w.top + (w.bottom if both else 0)
            mean = Fraction(w.total - drop, kept)
            out.append(as_text(mean))
    return out
```

```
if __name__ == "__main__":
    nums = [5, 1, 4, 2, 8, 3, 3, 9]
    fast = trimmed_window_means
    assert fast(nums, 3, 1) == brute(nums, 3, 1)
    assert fast(nums, 4, 2) == brute(nums, 4, 2)
    # a reduced fraction: (1 + 4) / 2 = 5/2
    assert fast([5, 1, 4], 3, 1) == ["5/2"]
    # duplicates at the boundary: remove exactly k copies
    dup = [3, 3, 3, 3]
    assert fast(dup, 3, 1) == ["3/1", "3/1"]
    # trimmed mean: drop k smallest and k largest
    both = fast(nums, 5, 1, True)
    assert both == brute(nums, 5, 1, True)
```

FUNCTION AND TEST CASES

Signature

```
def trimmed_window_means(nums: list[int], window: int, k:
int) -> list[str]:
```

Sample call

```
trimmed_window_means([1, 9, 2, 8], 3, 1)
```

Test cases

Input	Expected
`([1, 9, 2, 8], 3, 1)`	`['3/2', '5/1']`
`([-1, 1, 3], 2, 0)`	`['0/1', '2/1']`

ALSO REPORTED · 2 more

Sliding Window Average After Dropping the Largest Values | PracHub | Technical Screen

https://prachub.com/coding-questions/sliding-window-average-after-dropping-the-largest-values

Compute sliding-window average excluding top k | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-sliding-window-average-excluding-top-k

STUDY PATH: question 34 of 167 (Core 90)

Priority score 6.0 | Heaps and top-k | asked 28 days ago | frequency Low

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen | 3 of 7

Active Communication Top Users

ASKED 2026-09-22 | 28 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 228/228**

ABOUT THIS QUESTION

Topic: Core 90 - Heaps and top-k

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-22 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/active-communication-top-users

QUESTION

Process a continuous stream of user pairs such as **`A-B`** and **`B-C`**; each pair creates an active communication edge, and the system should output users ordered by current communication count. Follow-ups add self-communication handling, duplicate communication dedupe, and top-K users by count. An additional API variant exposes register and mostActiveUser(), then asks for constant-time most-active lookup and a changing-K ranking extension.SWEhashmapsetsortingtop-kstreamingmediumFrequencyLowLast asked2026-09-22Stagephone-screen

APPROACH

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a pair A-B counts for both users, whether repeated pairs count once, how self communication is treated, how ties order, and whether k changes over time.

BRUTE FORCE TO SAY OUT LOUD

Recount from all edges and sort users on every query. O(E log U).

OPTIMAL IDEA

Normalize pairs (sorted tuple). Keep an edge set for deduplication, a dict of user to count, and either a sorted structure of (count, user) or a bucket map count to ordered users so an update moves a user by one bucket. Top-K reads the highest buckets. Note the edge set can use quadratic space in the number of users; say so.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

A-B then B-A (duplicate), a self pair, ties on count, a new user, k larger than the number of users.

LIKELY FOLLOW-UPS

Constant-time most active user (track the maximum count with buckets), a changing k, or expiry of old edges.

PITFALLS

Claiming linear space while storing all pairs. Counting a duplicate edge twice.

HOW TO THINK ABOUT IT

Normalize the pair first (sorted tuple) so A-B and B-A are one edge, then keep an edge set for dedup and a count per user. The ranking part is a separate question: top-K comes from a heap, and the single most active user can be maintained in O(1) because counts only ever go up. Mention that the edge set uses quadratic space in the worst case.

Basis: Written. Assumed rules (confirm them): a pair 'A-B' is an unordered communication edge; repeated pairs count once; a user's activity is the number of distinct partners; order by activity descending, then name; self pairs (A-A) are ignored unless a flag says otherwise.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
pairs = set()
def add(a, b):
    pairs.add((min(a, b), max(a, b)))
def top_users():
    partners = defaultdict(set)
    for a, b in pairs:
        partners[a].add(b)
        partners[b].add(a)
    def rank(u):
        return (-len(partners[u]), u)
    return sorted(partners, key=rank)
```

TIME	SPACE
O(E + U log U) per query	**O(E)**

WHY IT IS SLOW

Each query rebuilds all counts and sorts, O(E + U log U). Maintaining counts incrementally with count buckets gives O(1) updates and an O(1) most-active lookup.

OPTIMAL SOLUTION · Edge set, counts per user, count buckets for the maximum

TIME	SPACE
O(1) per pair, O(U log k) for top-K, O(1) for the most active user	**O(E)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Self communication (A-A) should now count.

Pass a flag to the class: a self pair is one edge that adds one to that user's count (and only once). The dedup set already handles repeated self pairs.

FOLLOW-UP 2 Top-K with a changing K, or a fixed-size priority queue.

A heap of size K can be wrong after counts change, because an element outside it can overtake one inside. Re-run heapq.nsmallest on the count dict per query (O(U log K)) or maintain a sorted structure keyed by (count, name) updated on every increment.

FOLLOW-UP 3 The space claim.

Distinct pairs can number U^2 / 2 in the worst case, so the edge set is quadratic in the users; the counts alone are linear. If duplicates are not a concern, drop the set and keep O(U).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Active Communication Top Users
#
# Process a continuous stream of user pairs such as A-B
# and B-C; each pair creates an active communication edge,
# and the system should output users ordered by current
# communication count. Follow-ups add self-communication
# handling, duplicate communication dedupe, and top-K
# users by count. An additional API variant exposes
# register and mostActiveUser(), then asks for constant-
# time most-active lookup and a changing-K ranking
# extension.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Self communication (A-A) should now count.
# 2. Top-K with a changing K, or a fixed-size priority
# queue.
# 3. The space claim.
#
# INTERVIEW NOTES
# Normalize the pair first (sorted tuple) so A-B and B-A
# are one edge, then keep an edge set for dedup and a
# count per user. The ranking part is a separate question:
# top-K comes from a heap, and the single most active user
# can be maintained in O(1) because counts only ever go
# up. Mention that the edge set uses quadratic space in
# the worst case.
#
# BRUTE FORCE (say it first): Recompute counts from all
# edges on every query
# Keep every edge and, for each query, rebuild the
# distinct-partner counts and sort the users. Fine as an
# oracle; the work is repeated on each call. Time O(E + U
# log U) per query. Space O(E).


import heapq
from collections import Counter, defaultdict
```

```
# BRUTE FORCE (say it first): Recompute counts from all
# edges on every query | time O(E + U log U) per query
def _norm(pair):
    a, b = pair.split("-")
    return (a, b) if a <= b else (b, a)
```

```
class ActiveBrute:
    def __init__(self, count_self=False):
        self.edges, self.count_self = set(), count_self

    def add(self, pair):
        a, b = _norm(pair)
        if a == b and not self.count_self:
            return
        self.edges.add((a, b))

    def counts(self):
        c = Counter()
        for a, b in self.edges:
            c[a] += 1
            if a != b:
                c[b] += 1
        return c

    def top_k(self, k):
        c = self.counts()
        return sorted(c, key=lambda u: (-c[u], u))[:k]

    def most_active(self):
        c = self.counts()
        if not c:
            return set()
        m = max(c.values())
        return {u for u, v in c.items() if v == m}


# BASE QUESTION (optimal): Edge set, counts per user,
# count buckets for the maximum
# Idea: A new edge increments the count of both users.
# Buckets map each count to the set of users with it, so a
# user moves from bucket c to bucket c + 1.
# FOLLOW-UP 1: Self communication (A-A) should now count.
# Answer: Pass a flag to the class: a self pair is one
# edge that adds one to that user's count (and only once).
# The dedup set already handles repeated self pairs.
class Active:
    def __init__(self, count_self=False):
        # distinct edges seen so far (for the dedupe rule)
        self.edges = set()
        # user -> how many different partners
```

```
        self.count = Counter()
        # count -> users having exactly that count
        self.bucket = defaultdict(set)
        # the highest count right now (it never drops)
        self.top = 0
        self.count_self = count_self

    def bump(self, user):
        c = self.count[user]
        # move the user from bucket c to bucket c + 1
        if c:
            self.bucket[c].discard(user)
        self.count[user] = c + 1
        self.bucket[c + 1].add(user)
        self.top = max(self.top, c + 1)

    def add(self, a, b):
        # A-B and B-A are the same edge
        pair = (min(a, b), max(a, b))
        # follow-ups: skip self pairs (unless counted)
        # and skip a repeated pair
```

```
        skip_self = a == b and not self.count_self
        if skip_self or pair in self.edges:
            return
        self.edges.add(pair)
        self.bump(a)
        # a self pair counts once, not twice
        if a != b:
            self.bump(b)

    def most_active(self):
        # O(1): read the top bucket
        if not self.top:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        # k users by count, ties broken by name
        def rank(u):
            return (-self.count[u], u)
        return heapq.nsmallest(k, self.count, key=rank)


if __name__ == "__main__":
    a = Active()
```

```
    for x, y in [("A", "B"), ("B", "C"), ("B", "A")]:
        a.add(x, y)
    # B talks to A and C; the repeat B-A is ignored
    assert a.most_active() == {"B"}
    assert a.count["B"] == 2 and a.count["A"] == 1
    assert a.top_k(2) == ["B", "A"]
    # self pairs are ignored by default
    a.add("C", "C")
    assert a.count["C"] == 1
    # ... but can be switched on, counting once
    s = Active(count_self=True)
    s.add("C", "C")
    s.add("C", "C")
    assert s.count["C"] == 1
    assert Active().most_active() == set()
```

NOTES FROM CANDIDATES

A Java solution used an ordered map / tree structure to keep counts sortable.
The interviewer asked many clarification questions and expected dry runs.
The first version did not require self-communication handling; it was added as a coded follow-up.
Account for the per-user contact sets when explaining space use: the distinct relationship state can take quadratic space in the number of users. The interviewer explicitly challenged an initial linear-space claim.
A fixed-size priority-queue proposal did not finish the changing-K discussion; the interviewer asked for further optimization.
Maintaining the API variant
With registrations only and any tied maximum acceptable, normalize undirected pairs and store the distinct edges. For a new edge, increment each distinct endpoint once; an accepted self-edge changes one count, while an ignored self-edge changes none. Clarify that choice and the empty-state response.
Cache a user with the largest count. Counts never decrease, so only the endpoints of a new edge can displace the cached winner; duplicate edges do not change it. Reading this cache makes mostActiveUser() O(1), with expected O(1) registration work using

hash-based edge membership and counters under unit-cost user identifiers. A stricter tie rule requires separate handling.
For changing K, one option is a balanced ordered index of all users keyed by count and an agreed tie-breaker. Remove and reinsert the affected users when counts change. Updates cost O(log U); starting at the highest-ranked user and walking entries in descending-count order costs O(log U + K) for at most U returned users. This supports different K values without rebuilding a fixed-capacity heap and is a preparation option, not a specified interviewer solution.
Counters, the maximum cache, and the ranking index use O(U) space. Distinct relationship storage adds O(E), giving O(U + E) overall and O(U^2) in a dense graph.

PREPARATION

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

STUDY PATH: question 35 of 167 (Core 90)

Priority score 5.52 | Heaps and top-k | asked 78 days ago | frequency Single report

MEDIUM | IN APP LC 295 | DESIGN-STYLE (skippable) | 1Point3Acres | tech-screen | 4 of 7

Streaming Median (LC 295) with Follow-ups

ASKED 2026-08-03 | 78 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 50.4** | **30 DAYS 0** | **PRIORITY 69/228**

ABOUT THIS QUESTION

Topic: Core 90 - Heaps and top-k

Optimal answer from: your app's interview approach (LeetCode #295)

LeetCode match: LC 295 Find Median from Data Stream

Frequency: Single report | Last asked: 2026-08-03 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-median-with-follow-ups

QUESTION

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.SWEheapstreamingmediumFrequencySingle reportLast asked2026-08-03Stagetech-screen

APPROACH

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether values are integers, the range of values, how often median queries happen compared with inserts, and what to return for an even count (average of two middles).

BRUTE FORCE TO SAY OUT LOUD

Keep a sorted list and insert with bisect; median is the middle. O(n) per insert due to shifting.

OPTIMAL IDEA

Two heaps: a max-heap for the lower half and a min-heap for the upper half, sizes within one. Insert by pushing through one heap into the other and rebalancing. Median is the top of the larger heap or the average of both tops. O(log n) insert, O(1) median.

DRY-RUN THESE CASES

One element, two elements, duplicates, all equal, alternating very large and very small values.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Values in a small range (counting array), arbitrary k-th smallest (Fenwick tree or order-statistic tree), or too much data (quantile sketch).

PITFALLS

Rebalancing after the wrong heap push. Integer division when the median should be fractional.

HOW TO THINK ABOUT IT

Keep the data split at the median: a max-heap for the smaller half and a min-heap for the larger half, with sizes equal or differing by one. Then the median is read from the tops. Insert by pushing through one heap into the other to keep every smaller value on the left, and rebalance the sizes.

Basis: LeetCode 295, Find Median from Data Stream.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
nums = []  # always kept sorted
def add(x):
    insort(nums, x)
def median():
    n = len(nums)
    if n % 2:
        return nums[n // 2]
    return (nums[n // 2 - 1] + nums[n // 2]) / 2
```

TIME	SPACE
O(n) per insert (list shift), O(1) median	**O(n)**

WHY IT IS SLOW

Each insert shifts up to n elements, O(n) per add (fast in practice, but linear). Two heaps keep the median available with O(log n) per add and O(1) per query.

OPTIMAL SOLUTION · Two heaps

TIME	SPACE
O(log n) add, O(1) median	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

FOLLOW-UP 2 Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

FOLLOW-UP 3 What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

FOLLOW-UP 4 The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Streaming Median (LC 295) with Follow-ups
#
# A near-canonical LC 295 streaming-median implementation
# using two heaps, followed by two design extensions:
# exploit a very small value domain, then support an
# arbitrary k-th-smallest query instead of only the
# median.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Almost every value is in a small range (say 0 to
# 100). Can you do better than two heaps?
# 2. Now support an arbitrary k-th smallest, not only the
# median.
# 3. What if the data stream is too large to store?
# 4. The median of only the last K values.
#
# INTERVIEW NOTES
# Keep the data split at the median: a max-heap for the
# smaller half and a min-heap for the larger half, with
# sizes equal or differing by one. Then the median is read
# from the tops. Insert by pushing through one heap into
# the other to keep every smaller value on the left, and
# rebalance the sizes.
#
# BRUTE FORCE (say it first): Keep a sorted list and
# insert with bisect
# Insert each value into a sorted list with bisect.insort
# and read the middle. Very simple and the oracle; every
# insert shifts up to n elements, though in practice that
# is a fast memmove. Time O(n) per insert (list shift),
# O(1) median. Space O(n).


import bisect
import heapq
from bisect import bisect_left, insort
```

```python
# BRUTE FORCE (say it first): Keep a sorted list and
# insert with bisect | time O(n) per insert (list shift),
# O(1) median
class MedianSorted:
    def __init__(self):
        self.a = []
```

```python
    def addNum(self, x):
        bisect.insort(self.a, x)

    def findMedian(self):
        n = len(self.a)
        return (
            float(self.a[n // 2])
            if n % 2
            else (self.a[n // 2 - 1] + self.a[n // 2])
            / 2
        )


# BASE QUESTION (optimal): Two heaps
# Idea: Lower half in a max-heap, upper half in a min-
# heap, sizes differing by at most one and every lower
# value <= every upper value. The median is the larger
# heap's root, or the average of both roots.
class MedianFinder:
    def __init__(self):
        # low: max-heap (stored negated); high: min-heap
        self.low, self.high = [], []

    def add(self, x):
        # push through low so the halves stay ordered
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        # keep low the same size as high, or one bigger
        if len(self.high) > len(self.low):
            top = heapq.heappop(self.high)
            heapq.heappush(self.low, -top)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# FOLLOW-UP 1: Almost every value is in a small range (say
# 0 to 100). Can you do better than two heaps?
# Answer: Yes: a counting array gives O(1) inserts, and
# the median needs one pass over at most V counts, so O(V)
# per query with V fixed. Keep a moving pointer to the
# median bucket to make queries amortized O(1).
```

```python
class CountingMedian:
    """Follow-up: values are whole numbers in 0..maxv."""

    def __init__(self, maxv):
        self.counts = [0] * (maxv + 1)
        self.n = 0

    def add(self, x):
        self.counts[x] += 1
        self.n += 1

    def kth(self, k):
        # k-th smallest (1-based): one pass over the
        # counts
        seen = 0
        for value, c in enumerate(self.counts):
            seen += c
            if seen >= k:
                return value
```

```
    def median(self):
        lo = self.kth((self.n + 1) // 2)
        hi = self.kth(self.n // 2 + 1)
        return (lo + hi) / 2


# FOLLOW-UP 2: Now support an arbitrary k-th smallest, not
# only the median.
# Answer: Two heaps only work for one fixed rank. Use an
# order-statistics structure: a Fenwick tree over
# (compressed) values, or a balanced BST augmented with
# subtree sizes.
class Fenwick:
    """Follow-up: any k-th smallest in O(log V)."""

    def __init__(self, size):
        self.size, self.tree = size, [0] * (size + 1)

    def add(self, value):
        i = value + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def kth(self, k):
```

```
        # walk down the tree to the k-th smallest value
        pos, step = 0, 1 << self.size.bit_length()
        while step:
            nxt = pos + step
            if nxt <= self.size and self.tree[nxt] < k:
                pos = nxt
                k -= self.tree[nxt]
            step >>= 1
        return pos


# FOLLOW-UP 4: The median of only the last K values.
# Answer: Keep the window in a deque (to know what
# expires) and a sorted list (to read the middle); insert
# and remove with bisect. O(K) per update because of list
# shifting; two heaps with lazy deletion give O(log K).
class WindowMedian:
    """Follow-up: median of only the last k values."""

    def __init__(self, k):
        self.k, self.order, self.sorted = k, [], []

    def add(self, x):
        self.order.append(x)
        insort(self.sorted, x)
        if len(self.order) > self.k:
            old = self.order.pop(0)
            del self.sorted[bisect_left(self.sorted, old)]

    def median(self):
        s, n = self.sorted, len(self.sorted)
        return (s[(n - 1) // 2] + s[n // 2]) / 2


if __name__ == "__main__":
    m = MedianFinder()
    for x in [5, 2, 8]:
        m.add(x)
    assert m.median() == 5
    m.add(1)
    assert m.median() == 3.5
    c = CountingMedian(100)
    for x in [5, 2, 8, 1]:
        c.add(x)
    assert c.median() == 3.5
```

```
    f = Fenwick(100)
    for x in [5, 2, 8, 1]:
        f.add(x)
    ranks = [f.kth(k) for k in (1, 2, 3, 4)]
    assert ranks == [1, 2, 5, 8]
    w = WindowMedian(3)
    for x in [9, 1, 5, 7]:
        w.add(x)
    # window is [1, 5, 7]
    assert w.median() == 5
```

NOTES FROM CANDIDATES

Keep the lower half in a max-heap and the upper half in a min-heap. Maintain a size difference of at most one and keep every lower-half value no greater than every upper-half value.
For an even-sized stream, average the two heap roots.
Each insertion costs O(log n), a median query costs O(1), and the heaps use O(n) space.
For a small bounded domain, a frequency array gives O(1) updates and an O(U) order-statistic scan, where U is the domain size. For arbitrary k-th-smallest queries, use an order-statistic tree or a Fenwick/segment tree over a known or compressed domain for logarithmic update and selection.
The base problem required complete working code plus a dry run and complexity analysis.
The first follow-up stopped at design and pseudocode; the second was discussion-only because the round ran out of time.

PREPARATION

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

STUDY PATH: question 36 of 167 (Core 90)

Priority score 4.11 | Heaps and top-k | asked 139 days ago

MEDIUM | IN APP LC 621 | PracHub | Technical Screen | 5 of 7

Rearrange Tasks With Cooldown

POSTED 2026-06-03 | 139 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 92/228

ABOUT THIS QUESTION

Topic: Core 90 - Heaps and top-k

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #621 Task Scheduler

LeetCode match: LC 621 Task Scheduler (inferred from overview)

Posted on PracHub: 2026-06-03 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/rearrange-tasks-with-cooldown

QUESTION

Overview: This question evaluates task-scheduling and constraint-handling skills, testing algorithmic reasoning about arranging repeated jobs with cooldown constraints along with analysis of schedule optimality and algorithmic complexity.

APPROACH

Greedy with max-heap + cooldown queue, or the formula (maxFreq-1)*(n+1)+countOfMax vs len(tasks). Rearranged output: always take the available task with highest remaining count.

HOW TO THINK ABOUT IT

The most frequent task is the bottleneck. Lay it out with gaps of n between its copies; every other task can fill gaps. Total length is the larger of 'that skeleton' and 'just the number of tasks' (when there are so many tasks the gaps are all filled and no idle time is needed). The skeleton has (maxcount - 1) full blocks of size n + 1 plus one last block holding every task tied for the maximum.

Basis: LeetCode 621, Task Scheduler (minimum time), with LeetCode 358, Rearrange String k Distance Apart as the construction follow-up. Assumed rule: the same task needs n idle-or-other slots between runs.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(tasks, n):
    left = Counter(tasks)
    free_at = {}  # task -> time it is ready
    t = 0
    while any(left.values()):
        ready = [x for x in left if left[x]
                 and free_at.get(x, 0) <= t]
        if ready:
            task = max(ready, key=left.get)
            left[task] -= 1
            free_at[task] = t + n + 1
        t += 1  # idle if none ready
    return t
```

TIME	SPACE
O(T log k) where T is the answer	O(k)

WHY IT IS SLOW

It simulates every slot, including long idle stretches, O(T log k). The answer follows from the most frequent task: max(len(tasks), (maxCount - 1) * (n + 1) + number of tasks with that count).

OPTIMAL SOLUTION · Closed form from the most frequent task

TIME	SPACE
O(len(tasks))	O(1) for a fixed alphabet

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print an actual schedule, not just the time.

Use the heap-with-cooldown-queue construction: run the available task with the most copies left, park it for n slots, and emit an idle marker when nothing is available. Or rearrange a string so equal letters are at least k apart.

FOLLOW-UP 2 Different cooldowns per task.

The closed form no longer applies. Simulate with a heap, storing for each task the time it becomes available; pick the ready task with the most remaining copies, and jump the clock to the earliest availability when nothing is ready.

FOLLOW-UP 3 Tasks have priorities or deadlines.

Then it is a scheduling problem; say what you optimise (lateness, completion time) and use a heap ordered by deadline for the ready tasks.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Rearrange Tasks With Cooldown
#
# This question evaluates task-scheduling and constraint-
# handling skills, testing algorithmic reasoning about
# arranging repeated jobs with cooldown constraints along
# with analysis of schedule optimality and algorithmic
# complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print an actual schedule, not just the time.
# 2. Different cooldowns per task.
# 3. Tasks have priorities or deadlines.
#
# INTERVIEW NOTES
# The most frequent task is the bottleneck. Lay it out
# with gaps of n between its copies; every other task can
# fill gaps. Total length is the larger of 'that skeleton'
# and 'just the number of tasks' (when there are so many
# tasks the gaps are all filled and no idle time is
# needed). The skeleton has (maxcount - 1) full blocks of
# size n + 1 plus one last block holding every task tied
# for the maximum.
#
# BRUTE FORCE (say it first): Simulate each time slot,
# always running the available task with the most copies
# left
# At every time unit pick, among tasks not cooling down,
# the one with the most remaining copies; if none is
# available the slot is idle. A heap for ready tasks and a
# queue of tasks in cooldown make it work. Correct, but it
# simulates every slot, including long idle stretches.
# Time O(T log k) where T is the answer. Space O(k).


import heapq
from collections import Counter, deque


# BRUTE FORCE (say it first): Simulate each time slot,
# always running the available task with the most copies
# left | time O(T log k) where T is the answer
def brute(tasks, n):
    left = Counter(tasks)
    free_at = {}  # task -> time it is ready
    t = 0
```

```
    while any(left.values()):
        ready = [
            x
            for x in left
            if left[x] and free_at.get(x, 0) <= t
        ]
        if ready:
            task = max(ready, key=left.get)
            left[task] -= 1
            free_at[task] = t + n + 1
        t += 1  # idle if none ready
    return t


# BASE QUESTION (optimal): Closed form from the most
# frequent task
# Idea: Let m be the highest frequency and k the number of
# tasks that have that frequency. The skeleton needs (m -
# 1) * (n + 1) + k slots.
def least_interval(tasks, n):
```

```
    # closed form: the most frequent task sets the frame
    counts = Counter(tasks).values()
    most = max(counts, default=0)
    # how many tasks are tied for the most
    tied = sum(1 for c in counts if c == most)
    if not tasks:
        return 0
    # (most - 1) full frames of n + 1 slots, plus the last
    # row
    return max(len(tasks), (most - 1) * (n + 1) + tied)


# FOLLOW-UP 1: Print an actual schedule, not just the
# time.
# Answer: Use the heap-with-cooldown-queue construction:
# run the available task with the most copies left, park
# it for n slots, and emit an idle marker when nothing is
# available. Or rearrange a string so equal letters are at
# least k apart.
def schedule(tasks, n):
    # an actual schedule, with "idle" where nothing may
    # run
    heap = [(-c, t) for t, c in Counter(tasks).items()]
    heapq.heapify(heap)
    # tasks cooling down: (time ready, -left, task)
```

```
    cooling, out, now = deque(), [], 0
    while heap or cooling:
        # bring back tasks whose cooldown is over
        while cooling and cooling[0][0] <= now:
            _, left, task = cooling.popleft()
            heapq.heappush(heap, (left, task))
        if heap:
            # run the task with the most copies left
            left, task = heapq.heappop(heap)
            out.append(task)
            if left + 1 < 0:
                ready = now + n + 1
                cooling.append((ready, left + 1, task))
        else:
            out.append("idle")
        now += 1
    return out


# HELPERS for the checks below
def valid(plan, n):
    # equal tasks must be more than n slots apart
    last = {}
    for i, task in enumerate(plan):
        if task == "idle":
            continue
        if task in last and i - last[task] <= n:
            return False
        last[task] = i
    return True


if __name__ == "__main__":
    tasks = list("AAABBB")
    assert least_interval(tasks, 2) == 8
    assert least_interval(tasks, 0) == 6
    assert least_interval([], 3) == 0
    # many different tasks: no idle time needed
    assert least_interval(list("ABCDEFG"), 2) == 7
    plan = schedule(tasks, 2)
    assert len(plan) == 8 and valid(plan, 2)
    assert plan.count("idle") == 2
    for t, n in ((list("AAAABBC"), 3), (list("AB"), 5)):
        plan = schedule(t, n)
        assert valid(plan, n)
```

```
        assert len(plan) == least_interval(t, n)
```

FUNCTION AND TEST CASES

Signature

```
def solution(tasks: list[str], cooldown: int) -> str:
```

Sample call

```
solution(['A', 'A', 'A', 'B', 'B', 'C'], 2)
```

Test cases

Input	Expected
`(['A', 'A', 'A', 'B', 'B', 'C'], 2)`	`'ABCAB_A'`
`(['A', 'A', 'A', 'B'], 2)`	`'AB_A__A'`

ALSO REPORTED · 1 more

Design a task scheduler in Python. | Exponent

https://www.aced.io/questions/3504/design-task-scheduler-python

STUDY PATH: question 37 of 167 (Core 90)

Priority score 3.5 | Heaps and top-k | asked 89 days ago | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 6 of 7

Batch and Streaming Log Deduplication

ASKED 2026-07-23 | 89 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 226/228**

ABOUT THIS QUESTION

Topic: Core 90 - Heaps and top-k

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-07-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/batch-streaming-log-deduplication

QUESTION

Process **`(timestamp, message)`** logs by deduplicating on message and sorting retained entries by timestamp, first keeping the first-seen entry and then the latest timestamp. The streaming extension adds **`add(timestamp, message)`** and **`get_next()`**, requiring a map plus min-heap, lazy deletion, and a discussion of watermarks and bounded state.SWEdedupstreamingheapsortingmediumFrequencySingle reportLast asked2026-07-23Stageonsite-coding

APPROACH

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which entry to keep in each policy (first seen or latest timestamp), how ties on timestamp break, whether output is sorted by timestamp, and how the streaming interface returns results (get_next).

BRUTE FORCE TO SAY OUT LOUD

Sort all logs and dedupe with a set. Correct for the batch version.

OPTIMAL IDEA

Batch: keep a dict message to chosen entry (keep first-seen, or replace if the new timestamp is later) and sort the values by (timestamp, message). Streaming: a dict for membership plus a min-heap of (timestamp, message); on update of a message push a new entry and mark the old one stale (lazy deletion), and get_next pops until it finds a non-stale one. Amortized O(log n).

DRY-RUN THESE CASES

The same message twice at different times, equal timestamps, an empty stream, get_next on an empty queue.

LIKELY FOLLOW-UPS

Watermarks for late data, bounded memory with eviction by age.

PITFALLS

Returning a stale heap entry. Forgetting deterministic tie-breaking.

HOW TO THINK ABOUT IT

Dedup is a dictionary keyed by the message. Sorting is only for the output order. For a stream, a heap gives the smallest timestamp quickly, but when an entry is replaced you cannot delete from the middle of a heap, so leave the stale entry and skip it when it surfaces (lazy deletion).

Basis: Written. Assumed rules (confirm them): dedupe on the message; policy 'first' keeps the earliest in input order, policy 'latest' keeps the largest timestamp; output sorted by (timestamp, message); get_next returns the retained entry with the smallest timestamp and removes it.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(logs, keep_first=True):
    kept = []
    for entry in logs:
        same = [x for x in logs
                if x[1] == entry[1]]
        pick = max(same)
        if keep_first:
            pick = same[0]
        if entry == pick:
            kept.append(entry)
    return sorted(kept)  # by timestamp
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each line scans the others: O(n^2). A dictionary from message to the kept entry is one pass, and a heap with lazy deletion serves the streaming version.

OPTIMAL SOLUTION · Dictionary on the message, then sort

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Streaming: map + min-heap with lazy deletion

TIME	SPACE
O(log n) amortized per add and get_next	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Late data: how do you decide when it is safe to emit?

Use a watermark: only release entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness). get_next then refuses to return anything newer than the watermark, and anything arriving older than it is dropped or sent to a late-data side output.

FOLLOW-UP 2 Memory must stay bounded.

Evict messages older than a retention window (a second heap or the same heap keyed on timestamp) and accept that duplicates arriving after eviction are no longer detected. **Cost:** O(log n) per eviction with a heap; memory O(entries inside the retention window).

FOLLOW-UP 3 Keep the entry with the most occurrences instead.

Store (count, timestamp) per message and update the count on every duplicate; the heap key changes only if you rank by count, so push a new entry when the rank changes and rely on lazy deletion again.

FOLLOW-UP 4 Only release entries once they are safe (late data may still arrive).

Add a watermark: release only entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness), in order; anything older than the watermark that arrives later goes to a late-data path. The lazy-deletion heap pops stale entries as it goes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Batch and Streaming Log Deduplication
#
# Process (timestamp, message) logs by deduplicating on
# message and sorting retained entries by timestamp, first
# keeping the first-seen entry and then the latest
# timestamp. The streaming extension adds add(timestamp,
# message) and get_next(), requiring a map plus min-heap,
# lazy deletion, and a discussion of watermarks and
# bounded state.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Late data: how do you decide when it is safe to emit?
# 2. Memory must stay bounded.
# 3. Keep the entry with the most occurrences instead.
# 4. Only release entries once they are safe (late data
# may still arrive).
#
# INTERVIEW NOTES
# Dedup is a dictionary keyed by the message. Sorting is
# only for the output order. For a stream, a heap gives
# the smallest timestamp quickly, but when an entry is
# replaced you cannot delete from the middle of a heap, so
# leave the stale entry and skip it when it surfaces (lazy
# deletion).
#
# BRUTE FORCE (say it first): Quadratic scan for earlier
# or later duplicates
# For each log line, scan the rest (or the earlier lines)
# to see if it is the one to keep. Clear logic, quadratic
# cost. Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Quadratic scan for earlier
# or later duplicates | time O(n^2)
def dedupe_brute(logs, policy="first"):
    keep = []
    for i, (ts, msg) in enumerate(logs):
        if policy == "first":
            ok = all(
                logs[j][1] != msg for j in range(i)
            )
        else:
            ok = all(
                logs[j][1] != msg
                or logs[j][0] < ts
                or (logs[j][0] == ts and j < i)
                for j in range(len(logs))
                if j != i
            )
        if ok:
            keep.append((ts, msg))
    return sorted(keep, key=lambda e: (e[0], e[1]))


# BASE QUESTION (optimal): Dictionary on the message, then
# sort
# Idea: One pass with a dict from message to the retained
# timestamp (first seen: keep the first; latest: replace
# when the new timestamp is larger), then sort the
# retained entries for the output.
def dedupe(logs, policy="first"):
    # logs: (timestamp, message) pairs; keep one per
```

```
    # message
    kept = {}
    for ts, msg in logs:
        # "first" keeps the first seen, "latest" the
        # newest
        if msg not in kept or (
                policy == "latest" and ts > kept[msg]):
            kept[msg] = ts
    # retained entries sorted by time, then message
    return sorted((ts, msg) for msg, ts in kept.items())


# FOLLOW-UP 4: Only release entries once they are safe
# (late data may still arrive).
# Answer: Add a watermark: release only entries whose
# timestamp is at most (the largest timestamp seen minus
# the allowed lateness), in order; anything older than the
# watermark that arrives later goes to a late-data path.
class StreamDedup:
    """add(ts, msg) and get_next() with a lazy heap."""

    def __init__(self, policy="first", lateness=0):
        self.policy, self.lateness = policy, lateness
        # message -> the timestamp currently kept
        self.kept = {}
```

```
        # (timestamp, message), may hold outdated entries
        self.heap = []
        self.max_ts = None

    def watermark(self):
        # everything at or below this is safe to release
        if self.max_ts is None:
            return None
        return self.max_ts - self.lateness

    def add(self, ts, msg):
        wm = self.watermark()
        # too late: older than the watermark, drop it
        if wm is not None and ts < wm:
            return False
        if self.max_ts is None or ts > self.max_ts:
            self.max_ts = ts
        old = self.kept.get(msg)
        if old is None or (
                self.policy == "latest" and ts > old):
            self.kept[msg] = ts
            heapq.heappush(self.heap, (ts, msg))
        return True

    def get_next(self):
        wm = self.watermark()
        while self.heap:
            ts, msg = self.heap[0]
            # lazy deletion: skip entries that were
            # replaced
            if self.kept.get(msg) != ts:
                heapq.heappop(self.heap)
                continue
            # not safe yet: later data may still arrive
            if ts > wm:
                return None
            heapq.heappop(self.heap)
            del self.kept[msg]
            return ts, msg
        return None


if __name__ == "__main__":
    logs = [(5, "b"), (1, "a"), (9, "a"), (3, "b")]
    assert dedupe(logs, "first") == [(1, "a"), (5, "b")]
    assert dedupe(logs, "latest") == [(5, "b"), (9, "a")]
    s = StreamDedup("first", lateness=2)
    for ts, msg in [(1, "a"), (3, "c"), (5, "b")]:
        assert s.add(ts, msg)
    # watermark is 5 - 2 = 3: release 1, then 3
    assert s.get_next() == (1, "a")
    assert s.get_next() == (3, "c")
    # 5 is still above the watermark
    assert s.get_next() is None
    # data older than the watermark is dropped
    assert not s.add(2, "late")
    # a "latest" policy replaces the kept entry
    t = StreamDedup("latest")
    t.add(1, "x")
    t.add(4, "x")
    assert t.get_next() == (4, "x")
    assert t.get_next() is None
```

NOTES FROM CANDIDATES

Empty input, identical entries, equal timestamps, out-of-order arrivals, updates, and removals all need explicit semantics.
For keep-latest streaming behavior, stale heap entries must be detected against the current timestamp stored for that message.
Arbitrarily late arrivals make globally final timestamp ordering impossible without a watermark or maximum-lateness guarantee.

PREPARATION

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

STUDY PATH: question 38 of 167 (Core 90)

Priority score 7.97 | Heaps and top-k | asked 11 days ago | frequency Single report

HARD | IN APP LC 4 | 1Point3Acres | tech-screen | 7 of 7

Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 74.2** | **30 DAYS 61** | **PRIORITY 4/228**

ABOUT THIS QUESTION

Topic: Core 90 - Heaps and top-k

Optimal answer from: your app's interview approach (LeetCode #4)

LeetCode match: LC 4 Median of Two Sorted Arrays

Frequency: Single report | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/median-two-sorted-arrays-k-inputs

QUESTION

L5 R1 coding: LC 4 (Median of Two Sorted Arrays). Heading toward the LC 295 streaming two-heap approach drew a redirect from the interviewer. Follow-up: extend to many sorted inputs, in the spirit of LC 23 (Merge k Sorted Lists).

Per the candidate, the base question is LC 4 (Median of Two Sorted Arrays): given two sorted arrays, return the median of their combined elements.
Follow-up: the input becomes many sorted arrays instead of two, in the spirit of LC 23 (Merge k Sorted Lists).

APPROACH

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the median definition when the total is even (the mean of the two middle values), whether the arrays can be empty, whether duplicates are possible, and the sizes.

BRUTE FORCE TO SAY OUT LOUD

Merge and sort, then read the middle. O((m + n) log(m + n)); merging two sorted arrays with two pointers is O(m + n).

OPTIMAL IDEA

Binary search the cut in the shorter array: choose i in a and j = (m + n + 1) // 2 - i in b so the left parts hold half of the elements. The cut is valid when a[i-1] <= b[j] and b[j-1] <= a[i], with minus and plus infinity at the ends. The answer is the larger left value for an odd total, or the mean of the larger left and the smaller right value. O(log min(m, n)). For k arrays: binary search on the value and count elements at most x with a bisect per array, O(k log n log range), or pop from a min-heap until the middle, O(N log k).

DRY-RUN THESE CASES

One array empty, both of equal length, total odd and even, all elements equal, one array entirely smaller than the other, negative numbers.

LIKELY FOLLOW-UPS

Many sorted arrays (the stated follow-up), a stream (two heaps), or the k-th element instead of the median.

PITFALLS

Starting from the two-heap design of the streaming median (wrong family for static sorted input). Searching the longer array and running past its end. Off-by-one in the half size for odd totals.

HOW TO THINK ABOUT IT

Both inputs are already sorted, so the median is a partition problem, not a streaming one (the two-heap design of LeetCode 295 is the wrong family and the interviewer redirects it). Split the small array at i and the large one at j so that the left halves together hold half the elements; the split is right when every left value is at most every right value, which only needs two comparisons across the cut. That condition is monotone in i, so binary search it in O(log min(m, n)). For k arrays the same idea becomes: binary search on the value and count elements at most x with one bisect per array.

Basis: LeetCode 4, Median of Two Sorted Arrays; the follow-up to many sorted inputs is in the spirit of LeetCode 23, Merge k Sorted Lists.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def median(a, b):
    merged = sorted(a + b)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2
```

TIME	SPACE
O((m + n) log(m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that the inputs are sorted and costs O((m + n) log(m + n)) time and O(m + n) memory. A binary search for the partition reads only O(log min(m, n)) values.

OPTIMAL SOLUTION · Binary search on the partition

TIME	SPACE
O(log min(m, n))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Now there are k sorted arrays instead of two. How do you find the median?

Binary search on the answer value. For a candidate x, count the elements at most x with one bisect per array (O(k log n)); the median is the smallest x whose count reaches the middle rank. With integers the search range is the global min to max, so the cost is O(k log n log(range)). The heap alternative pops the smallest element from a k-way merge until it passes the middle, O((N / 2) log k), which is simpler but linear in the total size.

FOLLOW-UP 2 What is the heap version and when would you choose it?

A min-heap holds the front of every array. Pop (N + 1) / 2 times, remembering the one or two middle values. It is O(N log k) at worst, trivial to get right and needs no bound on the values, so choose it when the arrays are small or the values are not integers; choose the value search when N is huge and k is small.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Median of Two Sorted Arrays (LC 4) with a
# Follow-up Variant
#
# L5 R1 coding: LC 4 (Median of Two Sorted Arrays).
# Heading toward the LC 295 streaming two-heap approach
# drew a redirect from the interviewer. Follow-up: extend
# to many sorted inputs, in the spirit of LC 23 (Merge k
# Sorted Lists). Per the candidate, the base question is
# LC 4 (Median of Two Sorted Arrays): given two sorted
# arrays, return the median of their combined elements.
# Follow-up: the input becomes many sorted arrays instead
# of two, in the spirit of LC 23 (Merge k Sorted Lists).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Now there are k sorted arrays instead of two. How do
# you find the median?
# 2. What is the heap version and when would you choose
# it?
#
# INTERVIEW NOTES
# Both inputs are already sorted, so the median is a
# partition problem, not a streaming one (the two-heap
# design of LeetCode 295 is the wrong family and the
# interviewer redirects it). Split the small array at i
# and the large one at j so that the left halves together
# hold half the elements; the split is right when every
# left value is at most every right value, which only
# needs two comparisons across the cut. That condition is
# monotone in i, so binary search it in O(log min(m, n)).
# For k arrays the same idea becomes: binary search on the
# value and count elements at most x with one bisect per
# array.
#
# BRUTE FORCE (say it first): Merge everything and read
# the middle
# Concatenate, sort and take the middle one or two. It is
# the oracle and the baseline to say first. It ignores
# that the inputs are sorted, and it needs the whole
# merged array in memory. Time O((m + n) log(m + n)).
# Space O(m + n).


import bisect
import heapq


# BRUTE FORCE (say it first, it also checks the answers)
def median_brute(*arrays):
    # the oracle: merge, sort, read the middle
    merged = sorted(x for a in arrays for x in a)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Binary search on the partition
# Idea: Make a the shorter array. Binary search i in a,
# set j = half - i in b so the left side holds (m + n + 1)
# // 2 elements.
def median_two(a, b):
    """Both arrays are sorted: binary search the cut. The
    left parts must hold half of all the elements."""
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    if m + n == 0:
        raise ValueError("no elements")
    lo, hi, half = 0, m, (m + n + 1) // 2
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        # values just left and right of each cut
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < m else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < n else INF
        if a_left <= b_right and b_left <= a_right:
            # a valid cut: read the median off its edges
            if (m + n) % 2:
                return max(a_left, b_left)
            return (max(a_left, b_left)
                    + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1


# FOLLOW-UP 1: Now there are k sorted arrays instead of
# two. How do you find the median?
# Answer: Binary search on the answer value. For a
# candidate x, count the elements at most x with one
# bisect per array (O(k log n)); the median is the
# smallest x whose count reaches the middle rank.
```

```
def median_k_value(arrays):
    """k arrays: binary search the value; one bisect per
    array counts how many elements are at most x."""
    arrays = [a for a in arrays if a]
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")

    def kth(rank):
        # smallest x with at least `rank` elements <= x
        lo = min(a[0] for a in arrays)
        hi = max(a[-1] for a in arrays)
        while lo < hi:
            mid = (lo + hi) // 2
            count = sum(bisect.bisect_right(a, mid)
                        for a in arrays)
            if count >= rank:
                hi = mid
            else:
                lo = mid + 1
        return lo

    if total % 2:
        return kth(total // 2 + 1)
    return (kth(total // 2) + kth(total // 2 + 1)) / 2


# FOLLOW-UP 2: What is the heap version and when would you
# choose it?
# Answer: A min-heap holds the front of every array. Pop
# (N + 1) / 2 times, remembering the one or two middle
# values.
def median_k_heap(arrays):
    """k arrays, simple version: pop from a k-way merge
    until the middle one or two values have been seen."""
    heap = [(a[0], i, 0)
```

```
            for i, a in enumerate(arrays) if a]
    heapq.heapify(heap)
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")
    low_rank, high_rank = (total - 1) // 2, total // 2
    seen, middle = 0, []
    while heap:
        value, i, j = heapq.heappop(heap)
        if seen >= low_rank:
            middle.append(value)
        if seen >= high_rank:
            break
        seen += 1
        if j + 1 < len(arrays[i]):
            nxt = (arrays[i][j + 1], i, j + 1)
            heapq.heappush(heap, nxt)
    if total % 2:
        return middle[0]
    return (middle[0] + middle[1]) / 2


if __name__ == "__main__":
    assert median_two([1, 3], [2]) == 2
    assert median_two([1, 2], [3, 4]) == 2.5
    assert median_two([], [5]) == 5
    assert median_k_value([[1, 3], [2], [4, 5, 6]]) == 3.5
    assert median_k_heap([[1, 3], [2], [4, 5, 6]]) == 3.5
    import random
    for _ in range(300):
        arrays = []
        for _ in range(random.randint(1, 5)):
            size = random.randint(0, 6)
            arrays.append(sorted(random.randint(-9, 9)
                                 for _ in range(size)))
        if not any(arrays):
```

```
            continue
        want = median_brute(*arrays)
        assert median_k_value(arrays) == want
        assert median_k_heap(arrays) == want
        if len(arrays) >= 2:
            rest = sum(arrays[1:], [])
            rest.sort()
            assert median_two(arrays[0], rest) == want
```

NOTES FROM CANDIDATES

Pitfalls: Starting from the LC 295 (Find Median from Data Stream) two-heap design is the wrong family for the base question; the interviewer stepped in with a hint. Both inputs are already sorted — lean on that.

Follow-ups: Many sorted inputs instead of two

PREPARATION

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.